SESSION 21 CAPSTONE

ServiceDesk

A help desk ticketing platform, shipped through a complete DevOps pipeline

Aman Yadav · 24bcs10183 · 2nd Year · Batch A

What the application does

A support team lives and dies by two questions.
What is open right now, and what is about to breach its SLA.

- **Ticket**

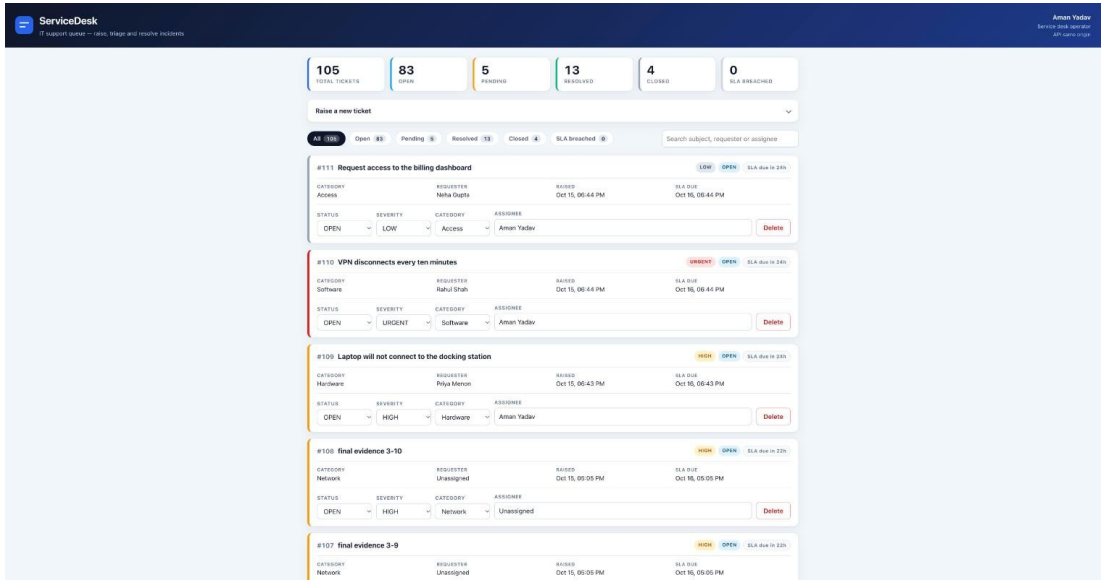
  subject, severity, status, requester, assignee

- **Category**

  unique name, referenced by tickets via a foreign key

- **SLA**

  due 24h after creation; breach counted only while open or pending



The console, served through the Kubernetes Ingress

11	2	2
ENDPOINTS	TABLES + FK	MIGRATIONS

How a change reaches production

1 **Commit** — git push to main → 2 **CI** — tests, lint, build → 3 **Scan** — Trivy gate → 4 **Registry** — GHCR, SHA tag → 5 **Helm** — upgrade release → 6 **Cluster** — rolling update

Observability and infrastructure sit either side of that chain

Prometheus + Grafana

scrapes the application's own /metrics; 23 panels on request rate, error ratio and latency

Terraform

VPC, subnets, security groups and an EKS cluster described as code

Helm

one chart, eleven revisions, install through rollback to image upgrade

Tests, and why they cannot touch real data

40	98%	5	100%
TESTS, ALL PASSING	COVERAGE	REQUIRED BY RUBRIC	ON app/main.py

The interesting part is isolation

- conftest.py resolves the test database before the application is imported
- If that URL ever resolves to the production DATABASE_URL it raises at import rather than running
- Every production row was md5-checksummed before and after a full run and came back identical
- The suite needs no PostgreSQL service in CI, so the pipeline stays fast

Containers

Backend

python:3.12-slim

Builder stage compiles wheels. pip, setuptools and wheel are deleted from the runtime image, which removed four fixable high-severity findings on its own.

Frontend

node build → nginx:alpine

Node compiles the bundle; only the built assets reach the final image. The compiler never ships.

Both images run as non-root — verified on the published artefact, not the local build

```
$ docker run --rm --entrypoint id ghcr.io/amanyadav7015/servicedesk-backend:5041ecb
  uid=10001(servicedesk) gid=10001(servicedesk)
```

docker compose up --build brings all three services up and is smoke-tested in CI on every push

The gate is real

A failing test did not just colour a badge red. It prevented an artefact from existing.

Proving the pipeline blocks a bad commit

I pushed a deliberately failing test on commit 759989a

Frontend build	success
Backend tests	FAILURE
Build, scan, publish	skipped
Compose stack	skipped

Skipped, not failed — the dependency graph stopped them before they took a runner

Checked by querying the registry directly, not by reading the pipeline log.

And the registry agrees

```
red commit   759989a  →  404
green commit d142944  →  200
14 tags, every one a commit SHA
no "latest" tag anywhere
```

Any running container traces back to the exact commit that produced it.

Security scanning, fixed at source

nginx:1.27-alpine

43

findings
41 high · 2 critical · all fixable

nginx:1.31-alpine + apk upgrade

0

findings
one minor version, zero suppressions

.trivyignore.yaml is empty — and that was a choice

- Two high-severity issues in the web framework could not be patched: another dependency pinned it below the fixed release.
- Rather than suppress them, I traced the constraint and upgraded the package holding the version down.
- The entire Python section of the report disappeared — removed at the source, not filtered out of view.

Kubernetes and Helm

5/5	2+2	11	1	4
PODS RUNNING	APP REPLICAS	HELM REVISIONS	INGRESS HOST	CLUSTERIP SVC

Eight real defects the chart had

- Ingress pointed at a service that did not exist, on a dead port
- The frontend could never have started — nginx proxies to a host no Service provided
- ServiceMonitor aborted the install where Prometheus CRDs were absent
- Database password sat in the Deployment instead of a Secret

helm lint passed with zero failures. Lint checks syntax, not wiring.

Proven, not asserted

```
POST a ticket through the Ingress → 201
delete BOTH backend pods
read the same ticket back → still there
confirmed as a row in PostgreSQL via psql
```

maxUnavailable: 0 means two image swaps never dropped a request.

Observability



23 panels on the application's own instrumentation, not node statistics

2/2

targets up, no scrape errors

9.87 ms

p95 latency under load

8

recording rules + 3 alerts

Two bugs I caught in my own queries: the fallback scrape job double-counted every series, and an SLO rule read a histogram bucket that does not exist, so it silently returned nothing.

Infrastructure as code — and what I could not do

Really applied, and destroyed

20 resources — VPC, two public subnets across two availability zones, private subnets, internet gateway, NAT, route tables, security groups. Verified with the AWS CLI, then destroyed and verified empty.

Planned only, never created

The EKS cluster and its managed node group. A genuine 35-resource plan with no errors — but the emulator I used does not implement the EKS API at all, and I have no AWS account.

I would rather lose the points than imply a cluster exists

The README says so in its first paragraph. No AWS console screenshot exists and none is claimed.

I also captured the control: with the skip flags off, the plan fails with "No valid credential sources found", and ~/.aws does not exist — proof the plan was not quietly run against a hidden account.

Passing your own tests is not the same as being portable

Three bugs that passed CI, passed the security scan, and ran fine in Kubernetes

Three bugs only a stricter host revealed

1 Container up, nothing listening

The stock nginx entrypoint derives a checksum by shelling out to apk, which blocks when run as a non-root user. The process stayed alive, so Docker called it healthy. No HTTP ever answered.

FIX

Removed the script this image replaces anyway

2 exec: Operation not permitted

The binary carried a capability so an unprivileged user could bind port 80. A host setting no_new_privileges refuses to execute a file with capabilities. Kubernetes allowed it; the stricter host did not.

FIX

Dropped the capability, moved to port 8080

3 502 with a healthy backend

The proxy forwarded the browser's Host header upstream, and a variable proxy_pass sends no SNI. A platform routing by Host with TLS termination could do neither.

FIX

Host $proxy_host, proxy_ssl_server_name on

None of the three is visible when the backend is plain HTTP on the same network — which is exactly why the cluster never surfaced them.

Where it stands

github.com/AmanYadav7015/devops-capstone-servicedesk

53
commits

40
tests

11
helm revisions

0
suppressed CVEs